Euler's Totient Function

$\varphi(n)$

for Competitive Programming

From zero to advanced: properties, proofs, reduction tricks,
22 contest problems and tested C++ code

What you will learn

- What $\varphi(n)$ means and four different ways to *see* it
- Every way to compute it: one number, a sieve, a range, a huge prefix sum
- All properties you need in contests, each with a short, easy reason
- Euler's theorem, inverses, huge exponents and power towers
- **How to turn a problem into** φ: gcd sums, lcm sums, coprime pairs, counting, cycles
- Advanced: Du sieve, inverse φ, Möbius link, primitive roots

Every formula in this guide was checked by brute force, and every C++ snippet was compiled and tested against a slow, obviously-correct solution.

October 2026

Contents

Part I — Foundations **5**

1 Warm-up: the tools you need **5**

1.1 Divisors . . . 5
1.2 gcd and "coprime" . . . 5
1.3 Prime factorization . . . 5
1.4 Modular arithmetic in one minute . . . 5
1.5 Two loop patterns with floors . . . 5

2 What is $\varphi(n)$? **6**

2.1 First examples . . . 6
2.2 What to notice in the table . . . 6
2.3 Four ways to think about $\varphi(n)$ . . . 6

3 How to compute $\varphi(n)$ **7**

3.1 The product formula . . . 7
3.2 One number: trial division, $O(\sqrt{n})$ . . . 7
3.3 All numbers up to N: the sieve . . . 7
3.4 The linear sieve, $O(N)$ . . . 8
3.5 A whole range $[L, R]$ with huge numbers . . . 8
3.6 Very large n . . . 9
3.7 Which method for which constraints? . . . 9

Part II — Properties **9**

4 The properties of φ **9**

5 Euler's theorem and everything you can do with it **13**

5.1 The theorem . . . 13
5.2 Modular inverse for any modulus . . . 13
5.3 Reducing a huge exponent . . . 13
5.4 When a and m are *not* coprime: the extended theorem . . . 14
5.5 Power towers $a_1^{a_2^{a_3^{\cdots}}} \bmod m$ . . . 14
5.6 Orders, primitive roots and cycles . . . 15
5.7 The Carmichael function (optional) . . . 15

Part III — Solving problems with φ **16**

6 The reduction toolbox **16**

7 Problem catalogue: 22 problems and their reductions **18**

Part IV — Advanced topics and cheat sheet **24**

8 Advanced topics **24**

8.1 The Du sieve: $\Phi(n)$ for n up to 10^{10} . . . 24

8.2 Inverse φ: find all n with $\varphi(n) = k$ . . . 25

8.3 When φ is not enough: the Möbius function . . . 26

8.4 Three convolution identities worth knowing . . . 27

8.5 Closed forms for multiplicative sums . . . 27

8.6 Very large numbers . . . 27

9 Cheat sheet **28**

9.1 All the formulas on one page . . . 28

9.2 Which code, and how fast? . . . 28

9.3 Mistakes to avoid . . . 29

9.4 How to attack a new problem . . . 29

9.5 Practice problems . . . 29

How to use this guide

This guide teaches **one function**, $\varphi(n)$, and then shows how many contest problems that look completely different are secretly the same φ problem in disguise.

- **Part I** (Sections 1–3): the basic idea, and all the ways to compute φ.
- **Part II** (Sections 4–5): every property, with a reason and an example; Euler's theorem.
- **Part III** (Sections 6–7): the heart of the guide. Eight reduction tricks, then 22 problems solved with them.
- **Part IV** (Sections 8–9): advanced topics and a two-page cheat sheet.

If you are new to the topic, read in order. If you are looking for a trick in the middle of a contest, jump to Section 6 (tricks), Section 7 (problems) or Section 9 (cheat sheet).

The coloured boxes

P# — A property

Blue boxes with a dark frame state a property (a formula you can use), followed by *why* it is true, an *example*, and where it is *used*.

Key idea

A key idea: the one thing to remember from a section.

Use in contests

How the idea appears in contests.

Careful!

A common mistake: an edge case, an overflow, or a condition that people forget.

Why is it true?

A short proof or intuition for a statement. Skip it on the first reading if you only want the result.

Notation

$a \mid b$	a divides b (b is a multiple of a)	$\lfloor x \rfloor$	floor (round down)
$\gcd(a, b)$	greatest common divisor	$\varphi(n)$	Euler's totient function
$\operatorname{lcm}(a, b)$	least common multiple	$\Phi(n)$	$\varphi(1) + \varphi(2) + \cdots + \varphi(n)$
coprime	$\gcd(a, b) = 1$	$\mu(n)$	Möbius function (Section 4, P11)
$\sum_{d \mid n}$	sum over all divisors d of n	$[P]$	1 if P is true, else 0
$a \equiv b \pmod{m}$	same remainder modulo m	$\omega(n)$	number of distinct primes of n

All C++ snippets assume this preamble (it is not repeated):

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
```

Part I — Foundations

1 Warm-up: the tools you need

You only need a few ideas from school mathematics. If all of this is familiar, skip to Section 2.

1.1 Divisors

We write $d \mid n$ when $n = d \cdot k$ for some integer k. The divisors of 12 are $1, 2, 3, 4, 6, 12$. Divisors come in pairs $(d, n/d)$, so we can list all of them in $O(\sqrt{n})$ time: loop $d = 1, 2, \ldots$ while $d \cdot d \le n$, and whenever $d \mid n$ take both d and n/d. A number $n \le 10^9$ has at most 1344 divisors; a number $n \le 10^{18}$ has at most 103680.

1.2 gcd and "coprime"

$\gcd(a, b)$ is the largest number that divides both a and b. Two numbers are **coprime** when $\gcd(a, b) = 1$, i.e. they share *no* prime factor. Euclid's algorithm computes gcd in $O(\log)$ time using $\gcd(a, b) = \gcd(b,\ a \bmod b)$. Three facts will be used again and again:

- **(G1)** $\gcd(ka, kb) = k \cdot \gcd(a, b)$. (Common factors can be pulled out.)
- **(G2)** $\gcd(a, n) = \gcd(a \bmod n,\ n)$. So *whether a is coprime to n depends only on a mod n*: the pattern repeats with period n.
- **(G3)** $\gcd(a, n) = \gcd(n - a,\ n)$. (Because $n - a \equiv -a \pmod{n}$.) So a and $n - a$ are either both coprime to n or both not.

1.3 Prime factorization

Every $n > 1$ can be written in exactly one way as $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ with distinct primes p_i. Example: $360 = 2^3 \cdot 3^2 \cdot 5$. Two numbers are coprime exactly when they have no prime in common.

How many *different* primes can n have? The product of the first nine primes is $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 \cdot 23 = 223\,092\,870$, and with ten primes it is already $6.4 \cdot 10^9$. So $n \le 10^9$ **has at most 9 distinct primes**, and $n \le 10^{18}$ has at most 15. This is why "loop over all subsets of the primes of n" (at most 2^{15} subsets) is cheap.

1.4 Modular arithmetic in one minute

$a \equiv b \pmod{m}$ means a and b have the same remainder when divided by m. Addition, subtraction and multiplication work as usual modulo m. *Division* is different: "dividing by a" means multiplying by the **inverse** a^{-1} (the number with $a \cdot a^{-1} \equiv 1$), and this inverse exists **if and only if** $\gcd(a, m) = 1$. This single sentence is the reason Euler's function matters for programming.

1.5 Two loop patterns with floors

Harmonic loop. Visiting every multiple of every $d \le N$ costs $N(\frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \ldots) \approx N \ln N$ steps:

```
for (int d = 1; d <= N; d++)
    for (int m = d; m <= N; m += d) { /* d divides m */ }
```

Block loop. The value $\lfloor n/d \rfloor$ takes at most $2\sqrt{n}$ different values as d goes from 1 to n. We handle all d with the same value at once:

```
for (ll l = 1, r; l <= n; l = r + 1) {
    ll q = n / l;       // floor(n/d) for every d in [l, r]
    r = n / q;          // last d that still gives the same q
```

```
    // use q and the whole block [l, r] together
}
```

Example for $n = 10$: the blocks are $[1, 1], [2, 2], [3, 3], [4, 5], [6, 10]$ with $q = 10, 5, 3, 2, 1$.

2 What is $\varphi(n)$?

Definition

$\varphi(n)$ is the number of integers k with $1 \le k \le n$ that are coprime to n:

$$\varphi(n) = \#\{\, k \in [1, n] \;:\; \gcd(k, n) = 1 \,\}.$$

2.1 First examples

$n = 12$**.** The primes of 12 are 2 and 3. Write $1, \ldots, 12$ and cross out every number that is divisible by 2 or 3:

$$\mathbf{1},\ 2,\ 3,\ 4,\ \mathbf{5},\ 6,\ \mathbf{7},\ 8,\ 9,\ 10,\ \mathbf{11},\ 12$$

(grey numbers share a factor with 12 and are crossed out). What remains is $1, 5, 7, 11$, so $\varphi(12) = 4$.

$n = 7$ **(a prime).** Nothing in 1..6 shares a factor with 7, and 7 itself does. So $\varphi(7) = 6$.

$n = 1$**.** $\gcd(1, 1) = 1$, so $\varphi(1) = 1$. (Careful: this is a special case that breaks many formulas.)

n	1	2	3	4	5	6	7	8	9	10
$\varphi(n)$	**1**	**1**	**2**	**2**	**4**	**2**	**6**	**4**	**6**	**4**
n	11	12	13	14	15	16	17	18	19	20
$\varphi(n)$	**10**	**4**	**12**	**6**	**8**	**8**	**16**	**6**	**18**	**8**
n	21	22	23	24	25	26	27	28	29	30
$\varphi(n)$	**12**	**10**	**22**	**8**	**20**	**12**	**18**	**12**	**28**	**8**

2.2 What to notice in the table

- For a prime p: $\varphi(p) = p - 1$ ($\varphi(7) = 6,\ \varphi(13) = 12,\ \varphi(29) = 28$).
- φ is *not* increasing: $\varphi(5) = 4$ but $\varphi(6) = 2$.
- For $n \ge 3$ the value $\varphi(n)$ is always even.
- $\varphi(2^k) = 2^{k-1}$: exactly half of the numbers up to 2^k are odd.

2.3 Four ways to think about $\varphi(n)$

Every property in this guide becomes easy if you keep these four pictures in mind. They all count the same thing.

View 1. Coprime numbers. Numbers in $[1, n]$ that share no prime with n.

View 2. Invertible residues. The remainders modulo n that have a multiplicative inverse. (Inverse exists $\iff$ coprime.)

View 3. Fractions in lowest terms. The fractions $\frac{k}{n}$ for $1 \le k \le n$ that cannot be reduced.

View 4. Generators of a cycle. If you walk around a circle of n points taking steps of size k, you visit every point exactly when $\gcd(k, n) = 1$. So there are $\varphi(n)$ "good" step sizes.

3 How to compute $\varphi(n)$

3.1 The product formula

The formula you will use 90% of the time

If $p_1, \ldots, p_k$ are the *distinct* primes dividing n, then

$$\varphi(n) = n\left(1 - \frac{1}{p_1}\right)\left(1 - \frac{1}{p_2}\right)\cdots\left(1 - \frac{1}{p_k}\right).$$

Intuition. A fraction $1/p$ of all numbers are multiples of p, so a fraction $(1 - 1/p)$ is *not*. Different primes behave independently, so we multiply. We start with n numbers and keep the fraction that survives each prime.

Example: $n = 360 = 2^3 \cdot 3^2 \cdot 5$**.** Distinct primes $2, 3, 5$:

$$\varphi(360) = 360 \cdot \tfrac{1}{2} \cdot \tfrac{2}{3} \cdot \tfrac{4}{5} = 96.$$

Only the *set* of primes matters; the exponents do not change the fraction.

Why it is true (inclusion–exclusion)

Take $n = 12$ with primes $2, 3$. Numbers in $[1, 12]$ divisible by 2: $12/2 = 6$. Divisible by 3: $12/3 = 4$. Divisible by both (i.e. by 6): $12/6 = 2$. Numbers sharing a factor: $6 + 4 - 2 = 8$ (we subtracted the double-counted ones). So the good ones: $12 - 6 - 4 + 2 = 4$. In general

$$\varphi(n) = \sum_{S \subseteq \{p_1..p_k\}} (-1)^{|S|} \frac{n}{\prod_{p \in S} p} = n \prod_i \left(1 - \frac{1}{p_i}\right).$$

The same sum will give us a way to count coprime numbers in any range (Problem Q4).

3.2 One number: trial division, $O(\sqrt{n})$

Factor n by trying $p = 2, 3, 4, \ldots$ up to $\sqrt{n}$. Each time we find a prime p, apply res $\leftarrow$ res $\cdot \frac{p-1}{p}$. We write this as `res -= res / p`, which stays in integers (at that moment p divides `res`, so the division is exact).

```
ll phi_single(ll n) {                  // O(sqrt n)
    ll res = n;
    for (ll p = 2; p * p <= n; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            res -= res / p;            // res = res * (p-1) / p  (exact)
        }
    }
    if (n > 1) res -= res / n;         // one leftover prime factor > sqrt
    return res;
}
```

Test: `phi_single(12)` = 4, `phi_single(360)` = 96, `phi_single(1)` = 1. Fast enough for a single n up to about 10^{12} (a loop of 10^6). For many queries use the sieves below.

3.3 All numbers up to N: the sieve

If we need φ for *every* number up to N (or for many queries), we compute them all together. The idea is the product formula applied from the prime's side: for every prime p, visit all its multiples j and apply the factor $(1 - 1/p)$.

```cpp
const int MAXN = 2000005;
int phi[MAXN];
void phi_sieve(int n) {              // O(n log log n)
    for (int i = 0; i <= n; i++) phi[i] = i;
    for (int i = 2; i <= n; i++)
        if (phi[i] == i)             // untouched so far => i is prime
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
}
```

How does it recognise a prime? `phi[i]==i` still holds only if no smaller prime has touched i, which means i is prime. The cost is $O(N \log \log N)$, practically linear.

3.4 The linear sieve, $O(N)$

The linear sieve builds φ by *multiplying a known value by one new prime*. Each composite number is generated exactly once, from $i \cdot p$ where p is its **smallest** prime factor. There are two cases for $\varphi(i \cdot p)$:

- $p \nmid i$: then p is a brand-new prime, $\gcd(i, p) = 1$, and by multiplicativity (P2 below) $\varphi(ip) = \varphi(i) \cdot (p-1)$.
- $p \mid i$: then p is already a prime factor of i, so the set of primes does not change and only n grows by the factor p: $\varphi(ip) = \varphi(i) \cdot p$.

```cpp
const int MAXN = 2000005;
int phi[MAXN], mu[MAXN];
bool composite[MAXN];
vector<int> primes;
void linear_sieve(int n) {           // O(n): every number is hit once
    phi[1] = mu[1] = 1;
    for (int i = 2; i <= n; i++) {
        if (!composite[i]) { primes.push_back(i); phi[i] = i - 1; mu[i] = -1; }
        for (int p : primes) {
            if ((ll)i * p > n) break;
            composite[i * p] = true;
            if (i % p == 0) {        // p already divides i
                phi[i * p] = phi[i] * p;
                mu[i * p] = 0;
                break;
            }
            phi[i * p] = phi[i] * (p - 1);   // p does not divide i
            mu[i * p] = -mu[i];
        }
    }
}
```

It also produces the list of primes and the Möbius function μ for free, which we need later. Typical limits: $N = 10^7$ takes about 0.1 s and roughly 9 bytes per number (two `int` arrays and a flag); $N = 10^8$ is usually too much memory.

3.5 A whole range $[L, R]$ with huge numbers

Sometimes the numbers are large (10^{12}) but the range is short ($\leq 10^6$ numbers). Then we can use a *segmented* version of the first sieve: only the primes up to $\sqrt{R}$ (at most 10^6) are needed. For every such prime p we visit its multiples inside $[L, R]$, apply the factor $(1 - 1/p)$ and divide p completely out of a copy of the number. What is left of a number after dividing out all small primes is 1 or one big prime, and that prime must still be applied.

```cpp
// phi(x) for every x in [L, R]; needs all primes up to sqrt(R)
vector<ll> phi_segment(ll L, ll R) {
    int len = R - L + 1;
    vector<ll> rem(len), res(len);
    for (int i = 0; i < len; i++) rem[i] = res[i] = L + i;
    for (int p : primes) {
        if ((ll)p * p > R) break;
        for (ll x = (L + p - 1) / p * p; x <= R; x += p) {
            int i = x - L;
            res[i] -= res[i] / p;
            while (rem[i] % p == 0) rem[i] /= p;
        }
    }
    for (int i = 0; i < len; i++)
        if (rem[i] > 1) res[i] -= res[i] / rem[i];   // leftover big prime
    return res;
}
```

Needs the `primes` vector from the linear sieve (built up to at least $\sqrt{R}$).

3.6 Very large n

For n up to 10^{18}, factor n with Pollard's rho algorithm and then use the product formula. Remember the underlying fact: *computing $\varphi(n)$ is exactly as hard as factoring n*. (This is why RSA is secure: knowing $\varphi(pq)$ lets you recover p and q.)

3.7 Which method for which constraints?

Situation	Method	Time
one $n \le 10^{12}$	trial division	$O(\sqrt{n})$
$n \le 10^{18}$	Pollard rho + formula	$\approx O(n^{1/4})$
all $n \le N$, $N \le 10^7$	linear sieve	$O(N)$
numbers $\le 10^{12}$, a window of $\le 10^6$	segmented sieve	$O(W \log\log R + \sqrt{R})$
prefix sum $\Phi(n)$ for $n \le 10^{10}$	Du sieve (Section 8.1)	$O(n^{2/3})$

Part II — Properties

4 The properties of φ

Here is the full list. Each property comes with a short reason, an example and (when useful) how it is used. They are numbered P1, P2, … so that later sections can refer to them.

P1 — Primes and prime powers

$$\varphi(p) = p - 1, \qquad \varphi(p^k) = p^k - p^{k-1} = p^{k-1}(p-1).$$

Why. In $1, \dots, p^k$ a number shares a factor with p^k exactly when it is a multiple of p. There are p^{k-1} such multiples $(p, 2p, \dots, p^k)$.
Example. $\varphi(81) = \varphi(3^4) = 81 - 27 = 54$. $\varphi(97) = 96$ since 97 is prime.
Use. For $n \ge 2$: $\varphi(n) = n - 1 \iff n$ is prime.

P2 — Multiplicative

If $\gcd(m, n) = 1$ then

$$\varphi(mn) = \varphi(m)\,\varphi(n).$$

Why. By the Chinese Remainder Theorem, every remainder x modulo mn corresponds to exactly one pair ($x \bmod m$, $x \bmod n$), and x is coprime to mn exactly when $x \bmod m$ is coprime to m *and* $x \bmod n$ is coprime to n. So the coprime remainders correspond to pairs of coprime remainders: $\varphi(m) \cdot \varphi(n)$ of them.
Example. $\varphi(12) = \varphi(4)\varphi(3) = 2 \cdot 2 = 4$.
Use. Computing φ from the factorization. More generally, a function with this property is called *multiplicative*; sums of a multiplicative function over the divisors of n are again multiplicative, so you can often work out a formula for prime powers only and combine (see Q7).

Careful!

P2 needs $\gcd(m, n) = 1$. For $m = n = 2$: $\varphi(2)\varphi(2) = 1$ but $\varphi(4) = 2$. For the general case see P4.

P3 — Product formula

$$\varphi(n) = \prod_{p^e \,\|\, n} p^{e-1}(p-1) = n \prod_{p \mid n} \Big(1 - \frac{1}{p}\Big).$$

Why. P1 gives $\varphi(p^e)$ and P2 multiplies the prime powers together.
Example. $\varphi(360) = \varphi(8)\varphi(9)\varphi(5) = 4 \cdot 6 \cdot 4 = 96$.
Use. The ratio $n/\varphi(n) = \prod_{p \mid n} \frac{p}{p-1}$ depends only on the set of primes, so $\varphi(n)/n$ is the same for n and $2n$ when n is even.

P4 — φ of a product when m and n are not coprime

Let $g = \gcd(m, n)$. Then

$$\varphi(mn) = \varphi(m)\,\varphi(n) \cdot \frac{g}{\varphi(g)}.$$

Special cases: $\varphi(n^k) = n^{k-1}\varphi(n)$; and if $m \mid n$ then $\varphi(mn) = m\,\varphi(n)$.
Why. The primes of mn are the primes of m together with those of n. In $\varphi(m)\varphi(n)$ the primes common to both (the primes of g) are counted twice; dividing by $\prod_{p \mid g}(1 - 1/p) = \varphi(g)/g$ fixes that.
Example. $m = 12,\ n = 18,\ g = 6$: $\varphi(12)\varphi(18) \cdot \frac{6}{\varphi(6)} = 4 \cdot 6 \cdot \frac{6}{2} = 72 = \varphi(216)$.

P5 — Parity

$\varphi(n)$ is even for all $n \geq 3$. Also

$$\varphi(2n) = \begin{cases} \varphi(n) & n \text{ odd}, \\ 2\varphi(n) & n \text{ even}. \end{cases}$$

Why. The coprime numbers come in pairs $k \leftrightarrow n - k$ (see P6); the two in a pair are different unless $k = n/2$, and $n/2$ is coprime to n only for $n = 2$. For $\varphi(2n)$: if n is odd, 2 is a new prime (P2: $\varphi(2) = 1$). If n is even, 2 is already a prime of n, so the factor is just multiplied by 2 (P4).
Use. Starting from any $n \geq 3$, the chain $n \to \varphi(n) \to \varphi(\varphi(n)) \to \cdots$ reaches 1 in about $\log_2 n$ steps: after the first step the value is even, and the φ of an even number is at most half of it. This is the engine of power towers (Section 5.5).

P6 — Symmetry: coprime numbers pair up

For $n \geq 2$: if $\gcd(k, n) = 1$ then also $\gcd(n - k, n) = 1$. Consequences:

$$\sum_{\substack{1 \leq k \leq n \\ \gcd(k,n)=1}} k = \frac{n\,\varphi(n)}{2}, \qquad \#\{(x, y) : \ x + y = n, \ x, y \geq 1, \ \gcd(x, y) = 1\} = \varphi(n).$$

Why. By (G3), $\gcd(k, n) = \gcd(n - k, n)$. Each pair $(k, n - k)$ adds up to n, and there are $\varphi(n)/2$ pairs. For the second formula note that $\gcd(x, y) = \gcd(x, x + y) = \gcd(x, n)$.
Example. $n = 12$: the coprime numbers are $1, 5, 7, 11$, with $1 + 11 = 5 + 7 = 12$, so the sum is $24 = 12 \cdot 4/2$. The pairs (x, y) with $x + y = 12$ are $(1, 11), (5, 7), (7, 5), (11, 1)$: four of them, $\varphi(12) = 4$.

Careful!

For $n = 1$ the sum is 1, not $\frac{1 \cdot 1}{2}$. The formula is for $n \geq 2$.

P7 — Gauss: the divisor sum

$$\sum_{d \mid n} \varphi(d) = n.$$

Why. Look at the n fractions $\frac{1}{n}, \frac{2}{n}, \ldots, \frac{n}{n}$ and reduce each to lowest terms. Each becomes $\frac{a}{d}$ with $d \mid n$ and $\gcd(a, d) = 1$. For a given divisor d the possible numerators a are exactly the numbers in $[1, d]$ coprime to d, so exactly $\varphi(d)$ fractions have reduced denominator d. Every one of the n fractions is counted once, hence $\sum_{d \mid n} \varphi(d) = n$.
Example. $n = 12$: $\varphi(1) + \varphi(2) + \varphi(3) + \varphi(4) + \varphi(6) + \varphi(12) = 1 + 1 + 2 + 2 + 2 + 4 = 12$.
Use. The engine behind every gcd-sum trick. Apply it to $n = \gcd(a, b)$ and you get $\gcd(a, b) = \sum_{d \mid a,\ d \mid b} \varphi(d)$ (Section 6, trick T2).

P8 — How many $k \leq n$ have $\gcd(k, n) = d$?

For every divisor d of n:

$$\#\{1 \leq k \leq n : \ \gcd(k, n) = d\} = \varphi(n/d).$$

Why. Such k are $k = d \cdot j$ with $1 \leq j \leq n/d$, and $\gcd(dj, n) = d \iff \gcd(j, n/d) = 1$.
Example ($n = 12$):

$d = \gcd(k, 12)$	1	2	3	4	6	12
those k	1,5,7,11	2,10	3,9	4,8	6	12
count $= \varphi(12/d)$	$\varphi(12) = 4$	$\varphi(6) = 2$	$\varphi(4) = 2$	$\varphi(3) = 2$	$\varphi(2) = 1$	$\varphi(1) = 1$

The counts add up to 12, which is P7 again, seen from the other side.
Use. It lets you rewrite any sum $\sum_{k=1}^{n} f(\gcd(k, n))$ as a sum over divisors (trick T1).

P9 — How big or small can $\varphi(n)$ be?

- $\varphi(n) \le n-1$ for $n \ge 2$, with equality exactly for primes.
- For composite n: $n - \varphi(n) \ge \sqrt{n}$. (A composite n has a prime factor $p \le \sqrt{n}$, and its $n/p \ge \sqrt{n}$ multiples are not coprime to n.)
- $\varphi(n) \ge \sqrt{n/2}$ always, and $\varphi(n) > \sqrt{n}$ for $n > 6$. So φ is never tiny.
- $n/\varphi(n) = \prod_{p|n} \frac{p}{p-1}$ grows only like $\log\log n$. It is largest when n is a product of the first primes: **for** $n \le 10^9$**:** $n/\varphi(n) < 6.12$**; for** $n \le 10^{18}$**:** $n/\varphi(n) < 7.21$**.**

Use. A random number is coprime to n with probability $\varphi(n)/n$, which is never below $1/7.21$ in contest ranges. So searches like "find a number coprime to n" end quickly, and the number of coprime numbers in an interval is always a decent fraction of its length.

P10 — Divisibility chain

If $m \mid n$ then $\varphi(m) \mid \varphi(n)$ (and so $\varphi(m) \le \varphi(n)$).
Why. By P3, $\varphi(n)/\varphi(m)$ is a product over the primes p of n of the integers p^{e-f} (if $p \mid m$, exponents $e \ge f \ge 1$) or $p^{e-1}(p-1)$ (if $p \nmid m$).
Example. $4 \mid 12$ and $\varphi(4) = 2 \mid \varphi(12) = 4$; $6 \mid 30$ and $\varphi(6) = 2 \mid \varphi(30) = 8$.

P11 — The Möbius form

Let $\mu(d) = 0$ if d has a repeated prime factor, otherwise $\mu(d) = (-1)^{\#\text{primes of } d}$ (so $\mu(1) = 1$, $\mu(2) = -1$, $\mu(6) = 1$, $\mu(4) = 0$). Then

$$\varphi(n) = \sum_{d|n} \mu(d)\,\frac{n}{d}, \qquad \text{and} \qquad [\gcd(a,b) = 1] = \sum_{d|\gcd(a,b)} \mu(d).$$

Why. It is the inclusion–exclusion sum of Section 3.1 written with d = the product of the chosen primes (sets of primes $\leftrightarrow$ squarefree numbers).
Example. $n = 12$: squarefree divisors $1, 2, 3, 6$ with $\mu = 1, -1, -1, 1$: $12 - 6 - 4 + 2 = 4$.
Use. When the φ-trick is not enough (Section 8.3): lcm of all pairs, coprime pairs in a rectangle.

P12 — The block identity

$$\sum_{d=1}^{n} \varphi(d)\left\lfloor \frac{n}{d} \right\rfloor = \frac{n(n+1)}{2}, \qquad \text{equivalently} \qquad \sum_{d=1}^{n} \Phi\left(\left\lfloor \frac{n}{d} \right\rfloor\right) = \frac{n(n+1)}{2}.$$

Why. Write each $m \le n$ as $m = \sum_{d|m} \varphi(d)$ (P7) and add over $m = 1..n$: the left side is $\sum m = \frac{n(n+1)}{2}$; on the right, $\varphi(d)$ is added once for each multiple of d up to n, i.e. $\lfloor n/d \rfloor$ times.
Example. $n = 6$: $1 \cdot 6 + 1 \cdot 3 + 2 \cdot 2 + 2 \cdot 1 + 4 \cdot 1 + 2 \cdot 1 = 21 = \frac{6 \cdot 7}{2}$.
Use. The Du sieve (Section 8.1) solves for $\Phi(n)$ in this identity to compute Φ for huge n.

P13 — Average size of φ

$$\Phi(n) = \sum_{k \le n} \varphi(k) \approx \frac{3}{\pi^2} n^2 \approx 0.304\, n^2.$$

Equivalently, two random integers are coprime with probability $6/\pi^2 \approx 60.8\%$.
Examples (computed with the code of this guide): $\Phi(10^6) = 303\,963\,552\,392$, $\Phi(10^9) = 303\,963\,551\,173\,008\,414$, $\Phi(10^{10}) = 30\,396\,355\,092\,886\,216\,366$.

Careful!

$\Phi(10^6) \approx 3 \cdot 10^{11}$ fits in `long long`, but $\Phi(10^{10}) \approx 3 \cdot 10^{19}$ does *not* (the limit is $9.2 \cdot 10^{18}$). For large n work modulo a prime, or use `__int128`.

P14 — Euler's theorem (details in Section 5)

If $\gcd(a, m) = 1$ then $a^{\varphi(m)} \equiv 1 \pmod{m}$. For a prime $m = p$ this is Fermat's little theorem $a^{p-1} \equiv 1$.

P15 — Orders and generators (details in Section 5.6)

The smallest $k > 0$ with $a^k \equiv 1 \pmod{m}$ is the *order* of a; it always divides $\varphi(m)$. Modulo a prime p, for each $d \mid p-1$ there are exactly $\varphi(d)$ numbers of order d; in particular there are $\varphi(p-1)$ primitive roots (numbers of order $p-1$).

5 Euler's theorem and everything you can do with it

5.1 The theorem

Euler's theorem

If $\gcd(a, m) = 1$ then $a^{\varphi(m)} \equiv 1 \pmod{m}$.

Examples. $m = 10$: $\varphi(10) = 4$ and $3^4 = 81 \equiv 1$. $m = 12$: $\varphi(12) = 4$ and $7^4 = 2401 = 200 \cdot 12 + 1 \equiv 1$.

Why is it true?

Let $r_1, \ldots, r_\varphi$ be the remainders coprime to m. Multiply each by a: the numbers $ar_1, \ldots, ar_\varphi$ are again coprime to m and are all different modulo m (if $ar_i \equiv ar_j$ we could cancel a, as a is invertible). So they are the same set of remainders in a different order. Hence $\prod(ar_i) \equiv \prod r_i$, i.e. $a^{\varphi(m)} \cdot R \equiv R$ where $R = \prod r_i$ is invertible. Cancel R.

5.2 Modular inverse for any modulus

Since $a \cdot a^{\varphi(m)-1} \equiv 1$, the inverse is

$$a^{-1} \equiv a^{\varphi(m)-1} \pmod{m} \qquad (\gcd(a, m) = 1).$$

For a prime m this is the familiar a^{m-2}. **Example:** $7^{-1} \bmod 12 = 7^3 = 343 = 28 \cdot 12 + 7 \equiv 7$, and indeed $7 \cdot 7 = 49 = 4 \cdot 12 + 1$.

```
ll power(ll a, ll e, ll m) {          // a^e mod m
    ll r = 1 % m; a %= m;
    while (e > 0) {
        if (e & 1) r = (__int128)r * a % m;
        a = (__int128)a * a % m;
        e >>= 1;
    }
    return r;
}
ll inv_mod(ll a, ll m) {              // needs gcd(a, m) = 1 ; m need NOT be prime
    return power(a, phi_single(m) - 1, m);
}
```

(The extended Euclidean algorithm is faster, $O(\log m)$, and does not need φ; the formula above is handy when you already know $\varphi(m)$, for example when m is a prime power.)

5.3 Reducing a huge exponent

If $\gcd(a, m) = 1$ the exponent can be reduced modulo $\varphi(m)$:

$$a^b \equiv a^{b \bmod \varphi(m)} \pmod{m}.$$

Example. $2^{100} \bmod 15$: $\varphi(15) = 8$, $100 \bmod 8 = 4$, so $2^{100} \equiv 2^4 = 16 \equiv 1$.

If b is given as a huge decimal string (say 10^5 digits), compute $b \bmod \varphi(m)$ digit by digit.

5.4 When a and m are *not* coprime: the extended theorem

Do not just reduce the exponent!

For non-coprime a, m the plain rule fails. Example: $2^4 \bmod 4 = 0$, but $\varphi(4) = 2$ and $4 \bmod 2 = 0$ would give $2^0 = 1$. Wrong!

Extended Euler

For **every** a and m, and every exponent $b \geq \log_2 m$ (any $b \geq 64$ is safe for $m < 2^{63}$):

$$a^b \equiv a^{(b \bmod \varphi(m)) + \varphi(m)} \pmod{m}.$$

Example. $2^{10} \bmod 12$: $\gcd(2, 12) = 2$. $\varphi(12) = 4$, $10 \bmod 4 + 4 = 6$, $2^6 = 64 \equiv 4$, and directly $2^{10} = 1024 = 85 \cdot 12 + 4$ ✓.

Why. Split m into the part sharing primes with a and the rest. Modulo a prime power p^e with $p \mid a$, the power a^b is already 0 as soon as $b \geq e$, and adding $\varphi(m)$ keeps the exponent that large. Modulo the part coprime to a, ordinary Euler applies.

For huge b given as a string, this gives (small exponents < 100 are handled exactly):

```
ll str_mod(const string& s, ll m) {            // string mod m, digit by digit
    ll r = 0;
    for (char c : s) r = ((__int128)r * 10 + (c - '0')) % m;
    return r;
}
// a^b mod m ; b = huge decimal string; any a, m
ll pow_huge(ll a, const string& b, ll m) {
    if (b.size() <= 2) return power(a, stoll(b), m);   // b < 100: exact
    ll ph = phi_single(m);                              // b >= 100 > log2(m)
    return power(a, str_mod(b, ph) + ph, m);            // a^(b mod phi + phi)
}
```

Tests: `pow_huge(2,"100",15)` $= 1$; $2^{10^{21}} \bmod 12 = 4$ (call it with b = the string `"1"` followed by 21 zeros).

5.5 Power towers $a_1^{a_2^{a_3^{\cdots}}} \bmod m$

The exponent of a_1 is $E = a_2^{a_3^{\cdots}}$, a monstrous number. By the extended theorem we only need $E \bmod \varphi(m)$ (and to know whether E is large). And $E \bmod \varphi(m)$ is itself a power tower, now modulo $\varphi(m)$. So we recurse down the chain

$$m \to \varphi(m) \to \varphi(\varphi(m)) \to \cdots \to 1,$$

which has only about $\log_2 m$ steps (P5). Once the modulus reaches 1, everything is 0 and we can stop: *the rest of the tower does not matter*. So for a tower of length 10^5 only the first ≈ 30 elements are used.

The "is it $\geq m$?" trick. We need to know whether the true exponent is at least $\varphi(m)$, but we only keep remainders. The standard solution defines

$$\mathrm{MOD}(x, m) = \begin{cases} x & x < m \\ x \bmod m + m & x \geq m \end{cases}$$

and uses it after *every* multiplication, so that a stored value is the true value if it is small, and "remainder plus m" (always $\geq m$) if the true value is large. The final answer is the stored value $\bmod m$.

```cpp
ll MOD(__int128 x, ll m) { return x < m ? (ll)x : (ll)(x % m) + m; }   // x mod m (+m if x >= m)
ll power_m(ll a, ll e, ll m) {      // a^e, remembers "was it >= m?"
    ll r = 1;
    while (e > 0) {
        if (e & 1) r = MOD((__int128)r * a, m);
        a = MOD((__int128)a * a, m);
        e >>= 1;
    }
    return r;
}
// a[i]^(a[i+1]^(a[i+2]^ ... ^a[n-1]))  (with the MOD trick) -- all a[i] >= 1
ll tower(const vector<ll>& a, int i, ll m) {
    if (m == 1 || a[i] == 1 || i + 1 == (int)a.size()) return MOD(a[i], m);
    return power_m(a[i], tower(a, i + 1, phi_single(m)), m);
}
// answer = tower(a, 0, m) % m
```

Test: $2^{3^{45}} \bmod (10^9+7) = 7\,513\,152$ (checked against Fermat: the exponent $3^{1024} \bmod (p-1)$ then $2^{(\cdot)} \bmod p$). For tiny towers: $2^{2^{2^2}} = 65536$, so `tower({2,2,2,2},0,1000)%1000`= 536. Used in Codeforces 906D (Power Tower).

5.6 Orders, primitive roots and cycles

The **order** $\mathrm{ord}_m(a)$ is the smallest $k > 0$ with $a^k \equiv 1 \pmod m$. By Euler's theorem it divides $\varphi(m)$. Modulo $m = 7$ ($\varphi = 6$):

a	1	2	3	4	5	6
$\mathrm{ord}_7(a)$	1	3	6	3	6	2

There is $\varphi(1) = 1$ element of order 1; $\varphi(2) = 1$ of order 2; $\varphi(3) = 2$ of order 3; $\varphi(6) = 2$ of order 6: exactly P15. The elements of order $6 = \varphi(7)$ are the **primitive roots**: their powers $g, g^2, \ldots, g^6$ visit every nonzero residue. Primitive roots exist modulo m exactly when $m \in \{1, 2, 4, p^k, 2p^k\}$ (p an odd prime), and then there are $\varphi(\varphi(m))$ of them. To find one for a prime p: factor $p-1$ and test candidates g with $g^{(p-1)/q} \not\equiv 1$ for all primes $q \mid p-1$.

```cpp
ll primitive_root(ll p) {           // p prime ; smallest generator of (Z/pZ)*
    ll ph = p - 1, n = ph;
    vector<ll> fac;                 // distinct primes of p-1
    for (ll q = 2; q * q <= n; q++)
        if (n % q == 0) { fac.push_back(q); while (n % q == 0) n /= q; }
    if (n > 1) fac.push_back(n);
    for (ll g = 2; g < p; g++) {
        bool ok = true;
        for (ll q : fac) if (power(g, ph / q, p) == 1) { ok = false; break; }
        if (ok) return g;           // order of g is exactly p-1
    }
    return 1;                       // p = 2
}
```

Test: `primitive_root(998244353)` = 3 (the famous NTT prime).

The same idea, in a circle. Take n points on a circle and jump k steps each time ($i \to i+k \bmod n$). The points split into $\gcd(k,n)$ cycles, each of length $n/\gcd(k,n)$. So the jump visits all points in one cycle exactly when $\gcd(k,n) = 1$: there are $\varphi(n)$ such jump sizes. In the cyclic group $\mathbb{Z}_n$ the element k has order $n/\gcd(n,k)$.

5.7 The Carmichael function (optional)

$\lambda(m)$ is the smallest exponent that works for *all* a coprime to m. It always divides $\varphi(m)$ and can be smaller: $\lambda(8) = 2$ while $\varphi(8) = 4$ (indeed $3^2, 5^2, 7^2 \equiv 1 \pmod 8$). Rule: $\lambda(p^k) = \varphi(p^k)$ except $\lambda(2^k) = 2^{k-2}$ for $k \geq 3$, and $\lambda(\text{coprime product}) = \text{lcm}$ of the parts (e.g. $\lambda(100) = \text{lcm}(\lambda(4), \lambda(25)) = \text{lcm}(2, 20) = 20$). Using λ

instead of φ is a rare optimisation; φ is always correct.

Part III — Solving problems with φ

6 The reduction toolbox

Contest problems almost never say "compute φ". They talk about gcd's, lcm's, fractions, cycles, or "how many numbers are coprime". The skill is to *recognise* the disguise. This section collects the eight moves that cover nearly everything. Section 7 then applies them to 22 problems.

T1 — Group by the gcd (a fixed n)

T1

For a fixed n, and any function f:

$$\sum_{k=1}^{n} f(\gcd(k,n)) \;=\; \sum_{d\mid n} f(d)\cdot\varphi\!\left(\frac{n}{d}\right).$$

Why. Put the numbers $k = 1..n$ into boxes according to $g = \gcd(k,n)$. By P8, box g contains exactly $\varphi(n/g)$ numbers, and every one of them contributes $f(g)$.
Example. $n = 12$, $f(g) = g$. The gcd's are $1,2,3,4,1,6,1,4,3,2,1,12$, which add up to 40. By the formula: $1\varphi(12) + 2\varphi(6) + 3\varphi(4) + 4\varphi(3) + 6\varphi(2) + 12\varphi(1) = 4 + 4 + 6 + 8 + 6 + 12 = 40$ ✓.
Gain. Instead of n gcd computations we need only the divisors of n.

T2 — Replace gcd by a sum of φ (two variables)

T2: the master trick

From P7 with $n = \gcd(a,b)$:

$$\gcd(a,b) = \sum_{d\mid a,\ d\mid b} \varphi(d).$$

Therefore, swapping the order of summation,

$$\sum_{a=1}^{n}\sum_{b=1}^{m} \gcd(a,b) = \sum_{d=1}^{\min(n,m)} \varphi(d)\cdot\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor.$$

Why the swap works. For a fixed d, the term $\varphi(d)$ is added once for every pair (a,b) with $d \mid a$ and $d \mid b$. There are $\lfloor n/d\rfloor$ multiples of d in $[1,n]$ and $\lfloor m/d\rfloor$ in $[1,m]$.
Example. $n = m = 6$: $\varphi(1..6) = 1,1,2,2,4,2$ and $\lfloor 6/d\rfloor = 6,3,2,1,1,1$, so the sum is $1\cdot 36 + 1\cdot 9 + 2\cdot 4 + 2\cdot 1 + 4\cdot 1 + 2\cdot 1 = 61$, which equals the brute-force total of all 36 gcd's.
Gain. With prefix sums Φ and the block loop of Section 1.5 this takes $O(\sqrt{n})$ per query.

T3 — Divide out the gcd

T3

$\gcd(a,b) = g$ if and only if $a = ga'$, $b = gb'$ with $\gcd(a',b') = 1$. So

$$\#\{(a,b)\in[1,n]\times[1,m] :\ \gcd(a,b) = g\} = \#\{\text{coprime pairs in } [1,\lfloor n/g\rfloor]\times[1,\lfloor m/g\rfloor]\}.$$

For $n = m$ this is $2\Phi(\lfloor n/g \rfloor) - 1$ (see T4).

Example. $n = 10,\ g = 2$: coprime pairs in $[1,5]^2$: $2\Phi(5) - 1 = 2 \cdot 10 - 1 = 19$ (and the brute force agrees).
Second proof of T2's example. $n = 6$: $\sum_g g\,(2\Phi(\lfloor 6/g \rfloor) - 1) = 1 \cdot 23 + 2 \cdot 7 + 3 \cdot 3 + 4 \cdot 1 + 5 \cdot 1 + 6 \cdot 1 = 61$ ✓.
Gain. Any sum of the form $\sum_{a,b} F(\gcd(a,b))$ becomes $\sum_g F(g)\,(2\Phi(\lfloor n/g \rfloor) - 1)$, for any F, e.g. $F = [g \text{ prime}]$ (Q12).

T4 — Coprime pairs are a prefix sum of φ

T4

The number of pairs $1 \le a \le b \le n$ with $\gcd(a,b) = 1$ equals $\Phi(n) = \varphi(1) + \cdots + \varphi(n)$.

Why. Fix b. The valid $a \le b$ are the numbers in $[1,b]$ coprime to b, which is $\varphi(b)$ of them (for $b = 1$: $a = 1$). Add over b.
Removing $(1,1)$ (the only pair with $a = b$ and gcd 1) gives the pairs with $a < b$: $\Phi(n) - 1$. Counting both orders: $2(\Phi(n) - 1) + 1 = 2\Phi(n) - 1$ ordered pairs.

$i \backslash j$	1	2	3	4	5	6
1	1	1	1	1	1	1
2	1	2	1	2	1	2
3	1	1	3	1	1	3
4	1	2	1	4	1	2
5	1	1	1	1	5	1
6	1	2	3	2	1	6

Table of $\gcd(i,j)$ for $1 \le i, j \le 6$. The green cells are coprime pairs: $23 = 2\Phi(6) - 1 = 2 \cdot 12 - 1$. The part above the diagonal has $\Phi(6) - 1 = 11$ green cells.

T5 — Fractions, Farey sequences, visible points

Counting pairs (a,b) with $\gcd(a,b) = 1$ is the same as counting other objects:

- **Fractions in lowest terms** a/b with $0 < a/b \le 1$ and $b \le n$: there are $\Phi(n)$ of them.
- The **Farey sequence** F_n (all reduced fractions in $[0,1]$ with denominator $\le n$) has $1 + \Phi(n)$ terms. Example: F_5 has $1 + 10 = 11$ terms: $\frac{0}{1}, \frac{1}{5}, \frac{1}{4}, \frac{1}{3}, \frac{2}{5}, \frac{1}{2}, \frac{3}{5}, \frac{2}{3}, \frac{3}{4}, \frac{4}{5}, \frac{1}{1}$.
- **Visible lattice points.** A point (x,y) with $1 \le x, y \le n$ is visible from the origin iff no other lattice point lies on the segment, i.e. iff $\gcd(x,y) = 1$: $2\Phi(n) - 1$ points.

T6 — Periodicity: counting coprime numbers in a range

By (G2), whether x is coprime to n depends only on x mod n, so the pattern repeats with period n and **every block of n consecutive numbers contains exactly $\varphi(n)$ coprime ones**. For the numbers in $[1,X]$:

$$\#\{x \le X : \gcd(x,n) = 1\} = \left\lfloor \frac{X}{n} \right\rfloor \varphi(n) + \#\{x \le X \bmod n :\ \gcd(x,n) = 1\}.$$

If n is huge, count directly by inclusion–exclusion over the (at most 15) distinct primes of n: $\sum_S (-1)^{|S|} \lfloor X / \prod S \rfloor$. See Q4.

T7 — Cycles, generators, necklaces

Jumping k steps on a circle of n points gives $\gcd(k,n)$ cycles of length $n/\gcd(k,n)$. This gives three kinds of statements:

- A jump of size k reaches everything $\iff$ $\gcd(k,n) = 1$: $\varphi(n)$ good k in $[0,n)$.
- In a cyclic group of size n, the number of elements of order exactly d is $\varphi(d)$.
- **Necklaces** (Burnside's lemma). Rotating a coloured necklace of n beads by k positions leaves it unchanged iff beads in each of the $\gcd(k,n)$ cycles have the same colour: $K^{\gcd(k,n)}$ colourings. Averaging over the n rotations:

$$\#\text{necklaces} = \frac{1}{n}\sum_{k=0}^{n-1} K^{\gcd(k,n)} = \frac{1}{n}\sum_{d\mid n} \varphi(d)\, K^{n/d},$$

by T1 (box g has $\varphi(n/g)$ rotations). Example: $n = 4, K = 2$: $\frac{1}{4}(16 + 4 + 2\cdot 2) = 6$.

T8 — Reduce the modulus

Whenever you see an inverse modulo a composite number, a huge exponent, or a power tower, think $\varphi(m)$ (Section 5).

Cheat table: pattern → trick

You see	Try
$\sum_{i\le n} f(\gcd(i,n))$, one n	T1 (divisors of n and φ)
$\sum_{i\le n}\sum_{j\le m}\gcd(i,j)$ or $\sum F(\gcd)$	T2 or T3, with prefix sums of φ
number of pairs with gcd $= g$ / gcd prime	T3
number of coprime pairs, fractions, visible points	T4, T5: $\Phi(n)$
numbers $\le X$ coprime to n; the k-th coprime number	T6
sum of the numbers coprime to n; $x + y = n$ with $\gcd(x,y) = 1$	P6
rotations, cycles, necklaces, primitive roots	T7
$a^b \bmod m$ with $\gcd(a,m) \neq 1$ or huge b; towers; inverses mod composite	T8 (Section 5)
φ(product) or a product of many numbers	P3 / P4

7 Problem catalogue: 22 problems and their reductions

Every problem below follows the same pattern: **Task** (what is asked), **Disguise** (why it is a φ problem), **Reduction** (the formula), **Example** (a small case you can check by hand), and then the tested code. Problems are ordered from easy to hard. Names of well-known online-judge problems of the same type are given so that you can practise.

Group A — Direct use

Q1 — Compute $\varphi(n)$ for one number

Task. Given n (up to 10^{12}), print $\varphi(n)$. Or: many test cases with $n \le 10^9$.
Reduction. Factor n, apply P3. Code: `phi_single` (Section 3.2).
Notes. For many tests with $n \le 10^9$, sieve the 3401 primes up to 31623 once and trial-divide only by those: about $3.4\cdot 10^3$ steps per test instead of $3\cdot 10^4$. **Check the problem's definition:** some statements count numbers *less than* n coprime to n; for $n = 1$ that gives 0, while $\varphi(1) = 1$.

Q2 — φ for all $n \le N$, many queries (SPOJ ETF)

Task. T queries, each asks $\varphi(n_i)$ with $n_i \le 10^6$ (or 10^7).
Reduction. Precompute with the linear sieve (Section 3.4), answer each query in $O(1)$. Store prefix sums Φ as well; almost every later problem needs them.

```
ll Phi[MAXN];                          // Phi[n] = phi(1) + ... + phi(n)
void build_prefix(int n) {
    for (int i = 1; i <= n; i++) Phi[i] = Phi[i - 1] + phi[i];
}
// ordered coprime pairs (a,b) in [1,n]^2 :  2*Phi[n] - 1
// coprime pairs with a<b             :  Phi[n] - 1
```

Q3 — Sum of the numbers coprime to n

Task. Print $\sum_{1 \le k \le n,\ \gcd(k,n)=1} k$ modulo $10^9 + 7$. (Variant: how many $k \le n$ are *not* coprime to n?)
Disguise. Coprime numbers come in pairs $(k, n-k)$ adding up to n (P6).
Reduction. Answer $= n\varphi(n)/2$ for $n \ge 2$ (and 1 for $n = 1$). Not coprime: $n - \varphi(n)$ numbers.
Example. $n = 12$: sum $= 12 \cdot 4/2 = 24$ (the numbers $1, 5, 7, 11$); non-coprime count $12 - 4 = 8$.
Careful. Do not divide by 2 after taking a remainder! Take the factor 2 out first: if n is even compute $(n/2) \cdot \varphi(n)$, otherwise $n \cdot (\varphi(n)/2)$ ($\varphi(n)$ is even for $n \ge 3$). Both are exact integers.

Q4 — Coprime numbers in $[1, X]$, and the k-th coprime number

Task. Given n and X (with $X \gg n$ allowed), count $x \in [1, X]$ with $\gcd(x, n) = 1$. Variant: find the k-th positive integer coprime to n.
Reduction (T6). Inclusion–exclusion over the distinct primes of n: $\sum_S (-1)^{|S|} \lfloor X / \prod S \rfloor$, at most 2^{15} terms. For the k-th: each block of n numbers holds $\varphi(n)$ coprime ones, so jump $\lfloor (k-1)/\varphi(n) \rfloor$ blocks, then binary search the r-th coprime number inside $[1, n]$ with the counting function.
Example. $n = 12$, $X = 100$: $100 - 50 - 33 + 16 = 33$. The 10-th coprime number to 12: blocks $\lfloor 9/4 \rfloor = 2$, remainder $r = 2$, the 2nd coprime in $[1, 12]$ is 5, so the answer is $2 \cdot 12 + 5 = 29$ (the sequence is $1, 5, 7, 11, 13, 17, 19, 23, 25, 29$).

```
ll count_coprime(ll X, ll n) {       // #x in [1,X] with gcd(x,n)=1, inclusion-exclusion
    vector<ll> ps;
    for (ll p = 2; p * p <= n; p++)
        if (n % p == 0) { ps.push_back(p); while (n % p == 0) n /= p; }
    if (n > 1) ps.push_back(n);
    int k = ps.size();
    ll res = 0;
    for (int mask = 0; mask < (1 << k); mask++) {
        ll prod = 1; int bits = 0;
        for (int i = 0; i < k; i++)
            if (mask >> i & 1) { prod *= ps[i]; bits++; }
        res += (bits & 1 ? -1 : 1) * (X / prod);
    }
    return res;
}
ll kth_coprime(ll k, ll n) {         // k-th positive integer coprime to n
    ll ph = phi_single(n);           // each block of n numbers holds exactly phi(n)
    ll full = (k - 1) / ph, r = (k - 1) % ph + 1;
    ll lo = 1, hi = n;
    while (lo < hi) {                // r-th coprime inside [1, n]
        ll mid = (lo + hi) / 2;
        if (count_coprime(mid, n) >= r) hi = mid; else lo = mid + 1;
    }
    return full * n + lo;
}
```

(For many queries with the same n, factor n once and reuse the prime list; the code above refactors for simplicity.)

Q5 — $\gcd(a, m) = \gcd(a + x, m)$

Task. Given a, m, count the $x \in [0, m)$ with $\gcd(a, m) = \gcd(a + x, m)$.
Reduction. Let $g = \gcd(a, m)$, $a = ga'$, $m = gm'$. For $\gcd(a + x, m) = g$ we need $g \mid x$, say $x = gy$ with $0 \le y < m'$; then $\gcd(g(a' + y), gm') = g \cdot \gcd(a' + y, m')$, which equals g iff $\gcd(a' + y, m') = 1$. As y runs through m' consecutive values, $a' + y$ runs through all residues modulo m', so exactly $\varphi(m')$ values work:

$$\text{answer} = \varphi\left(\frac{m}{\gcd(a, m)}\right).$$

Example. $a = 4, m = 12$: $g = 4$, answer $\varphi(3) = 2$ (the values $x = 0$ and $x = 4$).

Q6 — Split n into two coprime numbers

Task. Count the ways to write $n = x + y$ with $x, y \ge 1$ and $\gcd(x, y) = 1$.
Reduction. $\gcd(x, y) = \gcd(x, x + y) = \gcd(x, n)$, so x is any number in $[1, n - 1]$ coprime to n: $\varphi(n)$ **ordered ways**, i.e. $\varphi(n)/2$ unordered for $n \ge 3$.
Example. $n = 12$: $(1, 11), (5, 7)$ and their reverses.

Group B — gcd, lcm and pair counting

Q7 — $\sum_{i=1}^{n} \gcd(i, n)$ (Pillai's function)

Task. For a given n (or many $n \le 10^6$) compute $S(n) = \sum_{i=1}^{n} \gcd(i, n)$.
Reduction (T1 with $f(g) = g$). $S(n) = \sum_{d \mid n} d \cdot \varphi(n/d)$. Moreover S is multiplicative with

$$S(p^k) = (k + 1)p^k - k\,p^{k-1}.$$

Example. $S(12) = S(4) \cdot S(3) = (3 \cdot 4 - 2 \cdot 2)(2 \cdot 3 - 1) = 8 \cdot 5 = 40$, matching the direct sum $1 + 2 + 3 + 4 + 1 + 6 + 1 + 4 + 3 + 2 + 1 + 12 = 40$.
Code. For all $n \le N$ at once, loop over d and its multiples ($O(N \log N)$). For a single n up to about 10^9 use the generic helper that sums $f(d)\varphi(n/d)$ over the divisors, for *any* f. For a single huge n, factor once and use the closed form above.

```
ll P[MAXN];                          // P[m] = sum_{i=1..m} gcd(i, m)
void build_pillai(int n) {           // O(n log n)
    for (int d = 1; d <= n; d++)
        for (int m = d; m <= n; m += d)
            P[m] += (ll)d * phi[m / d];      // d | m, cofactor m/d
}
```

```
// sum_{i=1..n} f(gcd(i,n)) = sum_{d | n} f(d) * phi(n/d)
template <class F> ll sum_f_gcd(ll n, F f) {
    ll res = 0;
    for (ll d = 1; d * d <= n; d++) if (n % d == 0) {
        res += f(d) * phi_single(n / d);
        if (d != n / d) res += f(n / d) * phi_single(d);
    }
    return res;
}
```

Q8 — $\sum_{i=1}^{n} \operatorname{lcm}(i,n)$ (SPOJ LCMSUM)

Task. For up to $3 \cdot 10^5$ values of $n \le 10^6$, print $\sum_{i=1}^{n} \operatorname{lcm}(i,n)$.

Derivation. $\operatorname{lcm}(i,n) = \dfrac{i\,n}{\gcd(i,n)}$, so the sum is $n \sum_i \frac{i}{\gcd(i,n)}$. Group by $g = \gcd(i,n)$: write $i = gj$ with $\gcd(j, n/g) = 1$, $1 \le j \le n/g$; then $i/g = j$. Put $d = n/g$. The inner sum is the sum of the numbers $\le d$ coprime to d, which is $d\varphi(d)/2$ for $d \ge 2$ (P6) and 1 for $d = 1$. Hence

$$\sum_{i=1}^{n} \operatorname{lcm}(i,n) = \frac{n}{2}\Big(1 + \sum_{d \mid n} d\,\varphi(d)\Big).$$

(The "+1" repairs the $d = 1$ term, where $\frac{d\varphi(d)}{2}$ would be $\frac{1}{2}$ instead of 1.)

Example. $n = 12$: $d\varphi(d)$ for $d = 1, 2, 3, 4, 6, 12$ is $1, 2, 6, 8, 12, 48$, total 77; answer $\frac{12}{2} \cdot 78 = 468$. Direct: $12 + 12 + 12 + 12 + 60 + 12 + 84 + 24 + 36 + 60 + 132 + 12 = 468$.

Plan. Precompute $G[m] = \sum_{d \mid m} d\varphi(d)$ for all $m \le 10^6$ with the harmonic loop, then each query is $O(1)$. The largest answer ($n = 10^6$) is $277\,811\,686\,426\,000\,000 < 9.2 \cdot 10^{18}$, so `long long` is enough.

```
ll G[MAXN];                          // G[m] = sum_{d | m} d * phi(d)
void build_lcm(int n) {              // O(n log n)
    for (int d = 1; d <= n; d++)
        for (int m = d; m <= n; m += d)
            G[m] += (ll)d * phi[d];
}
ll lcm_sum(int n) {                  // sum_{i=1..n} lcm(i, n)
    return (ll)n * (G[n] + 1) / 2;   // fits in ll for n <= 10^6
}
```

Q9 — $\sum_{1 \le i < j \le N} \gcd(i,j)$ (UVa 11426, GCD Extreme II)

Task. Given N (up to a few million), compute the sum of $\gcd(i,j)$ over all pairs $i < j \le N$.

Two reductions.

(a) For a fixed j, $\sum_{i<j} \gcd(i,j) = S(j) - j$ by Q7 (the term $i = j$ gives j). So the answer is $\sum_{j=2}^{N} (S(j) - j)$; with a prefix array each query is $O(1)$.

(b) By T2: the sum over *all* ordered pairs is $A = \sum_d \varphi(d) \lfloor N/d \rfloor^2$; the diagonal contributes $\sum i = \frac{N(N+1)}{2}$; so the answer is $\dfrac{A - N(N+1)/2}{2}$.

Example. $N = 6$: $A = 61$ (Section 6, T2), diagonal 21, answer $(61 - 21)/2 = 20$. For $N = 10^6$ the answer is 4 071 628 673 912 (both methods agree).

```
ll sum_gcd_all_pairs(int n) {        // all ordered pairs: sum_d phi(d)*(n/d)^2
    ll ans = 0;
    for (int l = 1, r; l <= n; l = r + 1) {
        int q = n / l;
        r = n / q;                   // n/d is constant (= q) for d in [l, r]
        ans += (Phi[r] - Phi[l - 1]) * (ll)q * q;
    }
    return ans;
}
ll sum_gcd_i_lt_j(int n) {           // sum_{1<=i<j<=n} gcd(i,j)
    return (sum_gcd_all_pairs(n) - (ll)n * (n + 1) / 2) / 2;
}
```

Q10 — $\sum_{i\le n}\sum_{j\le m}\gcd(i,j)$, many queries

Task. T queries (n, m), each up to 10^6 or 10^7; print the sum of all $\gcd(i, j)$.
Reduction (T2). $\sum_d \varphi(d)\lfloor n/d\rfloor\lfloor m/d\rfloor$. Both quotients are constant on blocks, so using Φ the query is $O(\sqrt{n}+\sqrt{m})$.
Example. $n = m = 6$: 61.

```cpp
// sum_{i<=n,j<=m} gcd(i,j) = sum_d phi(d)*(n/d)*(m/d)
ll sum_gcd_rect(int n, int m) {
    ll ans = 0;
    int lim = min(n, m);
    for (int l = 1, r; l <= lim; l = r + 1) {
        r = min(n / (n / l), m / (m / l));      // both quotients constant on [l,r]
        ans += (Phi[r] - Phi[l - 1]) * (ll)(n / l) * (m / l);
    }
    return ans;
}
```

Q11 — Coprime pairs, reduced fractions, Farey sequences, visible points (POJ 2478, Project Euler 72)

Task. How many reduced fractions a/b with $0 < a/b < 1$ and $b \le n$ exist? How many pairs $1 \le a, b \le n$ are coprime? How many lattice points $(x, y) \in [1, n]^2$ are visible from the origin?
Reduction (T4, T5). Proper fractions: $\Phi(n) - 1$. Ordered coprime pairs / visible points: $2\Phi(n) - 1$. Pairs $a < b$: $\Phi(n) - 1$.
Example. $n = 6$: 23 coprime ordered pairs (the figure in Section 6). Project Euler 72 ($b \le 10^6$): $\Phi(10^6) - 1 = 303\,963\,552\,391$.

Q12 — Pairs with $\gcd = g$, and pairs with prime gcd (SPOJ PGCD)

Task. Count the pairs $(i, j) \in [1, n]^2$ whose gcd is exactly g (or is a prime).
Reduction (T3). Exactly g: $2\Phi(\lfloor n/g\rfloor) - 1$. Prime gcd: add this over all primes $p \le n$:

$$\sum_{p\le n,\ p\text{ prime}} \big(2\Phi(\lfloor n/p\rfloor) - 1\big).$$

Example. $n = 10$, $g = 2$: 19. Prime gcd: $p = 2 : 19$, $p = 3 : 2\Phi(3) - 1 = 7$, $p = 5 : 2\Phi(2) - 1 = 3$, $p = 7 : 1$; total 30.
Note. For a *rectangle* $n \ne m$ (as in SPOJ PGCD, $n, m \le 10^7$) the coprime-pair count in the rectangle needs the Möbius function (Section 8.3), and the real PGCD problem needs a faster summation over primes.

```cpp
ll pairs_gcd_prime(int n) {          // #(i,j) in [1,n]^2 with gcd(i,j) prime
    ll ans = 0;
    for (int p : primes) {
        if (p > n) break;
        ans += 2 * Phi[n / p] - 1;  // gcd exactly p  <=>  coprime pair in [1, n/p]
    }
    return ans;
}
```

Group C — Modular arithmetic

Q13 — Inverse modulo a composite number

Task. Print a^{-1} mod m where m is not prime and $\gcd(a, m) = 1$.
Reduction. $a^{-1} \equiv a^{\varphi(m)-1}$ (Section 5.2; code `inv_mod`). **Example:** 7^{-1} mod 12 = 7; 5^{-1} mod 36 = 29 (check: $5 \cdot 29 = 145 = 4 \cdot 36 + 1$).

Q14 — $a^b \bmod m$ with a gigantic b

Task. b has up to 10^5 digits, $m \le 10^9$ and a, m may share factors.
Reduction. Extended Euler (Section 5.4): $b \to (b \bmod \varphi(m)) + \varphi(m)$; code `pow_huge`. **Example:** $2^{100} \bmod 15 = 1$; $2^{10^{21}} \bmod 12 = 4$.

Q15 — Power tower (Codeforces 906D)

Task. Given an array a and queries $[l, r]$, compute $a_l^{a_{l+1}^{\cdot^{\cdot^{a_r}}}} \bmod m$.
Reduction. Recurse down the chain $m \to \varphi(m) \to \cdots \to 1$ with the "MOD" trick (Section 5.5). At most ≈ 30 array elements are used per query because the modulus reaches 1.
Speed-up. Compute the chain $m, \varphi(m), \varphi(\varphi(m)), \ldots$ once at the start and reuse it for all queries, instead of calling `phi_single` inside the recursion.

Q16 — Necklaces, rotations and cyclic arrays

Task. Count the colourings of a circle of n beads with K colours, where rotations are considered the same (up to a prime modulus), n up to 10^9.
Reduction (T7). $\frac{1}{n}\sum_{d|n} \varphi(d) K^{n/d}$. Enumerate the divisors of n in $O(\sqrt{n})$, compute φ of each, use fast power and one modular inverse of n (valid since $n <$ the prime).
Example. $n = 4, K = 2$: 6 necklaces. $n = 6, K = 3$: 130. Also: a rotation by k is a single cycle iff $\gcd(k, n) = 1$, so there are $\varphi(n)$ such rotations.

```
// necklaces of n beads, k colours, mod MODP (n < MODP)
ll necklaces(ll n, ll k) {
    ll sum = 0;
    for (ll d = 1; d * d <= n; d++) {
        if (n % d) continue;
        sum = (sum + phi_single(d) % MODP * power(k, n / d, MODP)) % MODP;
        if (d != n / d)
            sum = (sum + phi_single(n / d) % MODP * power(k, d, MODP)) % MODP;
    }
    return sum * power(n % MODP, MODP - 2, MODP) % MODP;   // divide by n
}
```

Group D — Advanced (solved in Sections 3.5 and 8)

Q17 — $\Phi(n)$ for n up to 10^{10}

Sum of $\varphi(1), \ldots, \varphi(n)$ modulo a prime, or exact coprime-pair counts for huge n. **Tool:** Du sieve (Section 8.1), built on the block identity P12.

Q18 — $\varphi(x)$ for every x in a window $[L, R]$, $R \le 10^{12}$

Tool: segmented sieve (Section 3.5, `phi_segment`). For example "sum of $\varphi(x)$ for $x \in [10^{12} - 10^5, 10^{12}]$".

Q19 — All n with $\varphi(n) = k$

Count them or find the smallest. **Tool:** inverse φ search over the primes p with $(p - 1) \mid k$ (Section 8.2).

Q20 — $\sum_{i,j \le n} \operatorname{lcm}(i, j)$

Tool: Section 8.3: the φ-trick is not enough here; a Möbius sum combined with T3 solves it in about $O(n^{3/4})$.

Q21 — φ of the product of a range, with updates (Codeforces 1114F)

Task. The array has small values ($a_i \le 300$), with range-multiply updates and queries for $\varphi(a_l \cdot a_{l+1} \cdots a_r) \bmod 10^9 + 7$.
Reduction. Use P3: $\varphi\left(\prod a_i\right) = \prod a_i \cdot \prod_{p \in \text{primes present}} \frac{p-1}{p}$. Only the *set* of primes that divide the product matters, and there are only 62 primes below 300, so the set fits in a 64-bit mask. A lazy segment tree stores the product modulo $10^9 + 7$ and the OR of the masks; the answer multiplies the product by $(p-1) \cdot p^{-1}$ for each bit set.

Q22 — Project Euler style: extremes of $n/\varphi(n)$

PE 69: maximise $n/\varphi(n)$ for $n \le 10^6$. Since $n/\varphi(n) = \prod_{p \mid n} \frac{p}{p-1}$ grows with every new small prime, the best n is a product of the first primes: $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 = 510510$.
PE 70: minimise $n/\varphi(n)$ for $n < 10^7$ subject to $\varphi(n)$ being a digit permutation of n. To make $n/\varphi(n)$ small use as few, as large primes as possible: two primes near $\sqrt{10^7}$. The answer is $8\,319\,823 = 2339 \cdot 3557$ with $\varphi = 8\,313\,928$ (all verified by the code of this guide).

Part IV — Advanced topics and cheat sheet

8 Advanced topics

8.1 The Du sieve: $\Phi(n)$ for n up to 10^{10}

A linear sieve stops at about 10^7–10^8. To get $\Phi(n) = \varphi(1) + \cdots + \varphi(n)$ for $n = 10^{10}$ we use the block identity P12:

$$\sum_{d=1}^{n} \Phi\left(\left\lfloor \frac{n}{d} \right\rfloor\right) = \frac{n(n+1)}{2}.$$

The term $d = 1$ is $\Phi(n)$ itself. Move the others to the right:

$$\boxed{\Phi(n) = \frac{n(n+1)}{2} - \sum_{d=2}^{n} \Phi\left(\left\lfloor \frac{n}{d} \right\rfloor\right)}$$

The right side needs Φ only at the points $\lfloor n/d \rfloor$, and there are just $O(\sqrt{n})$ different such points (the block loop of Section 1.5). Compute them recursively and **memoize**. Small arguments ($\le L$) are read from a table made by the linear sieve. If $L \approx n^{2/3}$ the total time is $O(n^{2/3})$.

Test values (to debug your own implementation; all produced by the code below and confirmed by an independent method through the Mertens function):

n	$\Phi(n)$ exactly	$\Phi(n) \bmod (10^9 + 7)$
10^6	303 963 552 392	963 550 271
10^9	303 963 551 173 008 414	45 263 571
10^{10}	30 396 355 092 886 216 366	111 732 206

On the machine used to prepare this guide, $n = 10^9$ took about 0.1 s and $n = 10^{10}$ about 1 s with a sieve limit $L = 2 \cdot 10^6$ (increase L to a few million for 10^{10}–10^{11}). The exact values exceed the `long long` range for $n = 10^{10}$, which is why the code works modulo a prime.

```cpp
const ll MODP = 1000000007;
const int LIM = 2000000;            // sieve limit; use about n^(2/3)
unordered_map<ll, ll> memo;
ll tri(ll n) {                      // n(n+1)/2 mod MODP, safe for n up to 10^18
    ll a = n, b = n + 1;
    if (a % 2 == 0) a /= 2; else b /= 2;
    return (a % MODP) * (b % MODP) % MODP;
}
ll Phi_du(ll n) {                   // phi(1)+...+phi(n)  mod MODP, n up to ~10^10
    if (n <= LIM) return Phi[n] % MODP;         // Phi[] from build_prefix(LIM)
    auto it = memo.find(n);
    if (it != memo.end()) return it->second;
    ll res = tri(n);                            // sum_{d=1..n} Phi(n/d) = n(n+1)/2
    for (ll l = 2, r; l <= n; l = r + 1) {
        r = n / (n / l);
        res = (res - (r - l + 1) % MODP * Phi_du(n / l)) % MODP;
    }
    if (res < 0) res += MODP;
    return memo[n] = res;
}
```

The general Du-sieve idea. If $h = f * g$ (Dirichlet convolution: $h(k) = \sum_{d|k} g(d) f(k/d)$) and the prefix sums of g and h are easy, then $S_f(n) = S_h(n) - \sum_{d=2}^{n} g(d) S_f(\lfloor n/d \rfloor)$ (for $g(1) = 1$). For φ: $g = 1$ (constant), $h = \mathrm{id}$, because $\sum_{d|k} \varphi(d) = k$ (P7). For the Möbius function: $g = 1$, $h = [k = 1]$.

8.2 Inverse φ: find all n with $\varphi(n) = k$

Observations.

- If $p^e \,\|\, n$ then $\varphi(p^e) = p^{e-1}(p-1)$ divides $\varphi(n) = k$. So every prime of n satisfies $(p-1) \mid k$. Find all divisors d of k for which $d+1$ is prime: at most 1344 candidates for $k \le 10^9$.
- Use the candidate primes in increasing order and choose an exponent for each one (or skip it), dividing k by $\varphi(p^e)$ until it becomes 1: a depth-first search.
- For odd $k > 1$ there is no solution (P5). Some even k also have none, e.g. $k = 14$.

Examples. $\varphi(n) = 8$: $n \in \{15, 16, 20, 24, 30\}$. $\varphi(n) = 5 \cdot 10^8$ has 106 solutions (each one verified with `phi_single`).

```cpp
bool is_prime(ll x) {
    if (x < 2) return false;
    for (ll d = 2; d * d <= x; d++) if (x % d == 0) return false;
    return true;
}
vector<ll> cand, sols;
void dfs_inv(int i, ll rem, ll cur) {
    if (rem == 1) sols.push_back(cur);
    for (int j = i; j < (int)cand.size(); j++) {
        ll p = cand[j];
        if (rem % (p - 1) != 0) continue;
        ll r = rem / (p - 1), c = cur * p;       // use p^1 : phi = p-1
        while (true) {
            dfs_inv(j + 1, r, c);
            if (r % p != 0) break;
            r /= p; c *= p;                      // raise exponent: phi gains factor p
        }
    }
}
vector<ll> inverse_phi(ll k) {       // all n with phi(n) = k, sorted
    cand.clear(); sols.clear();
    for (ll d = 1; d * d <= k; d++) if (k % d == 0) {    // p-1 must divide k
        if (is_prime(d + 1)) cand.push_back(d + 1);
        if (d != k / d && is_prime(k / d + 1)) cand.push_back(k / d + 1);
    }
    sort(cand.begin(), cand.end());
    dfs_inv(0, k, 1);
    sort(sols.begin(), sols.end());
    return sols;
}
```

8.3 When φ is not enough: the Möbius function

Two situations need μ (P11): *rectangles* where the trick T4 does not apply, and *lcm* sums over pairs.

Coprime pairs in a rectangle. Using $[\gcd(i,j)=1] = \sum_{d\mid\gcd(i,j)} \mu(d)$ and swapping sums, as in T2:

$$\#\{(i,j) \in [1,n]\times[1,m] : \gcd(i,j)=1\} = \sum_{d} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor \left\lfloor \frac{m}{d} \right\rfloor .$$

For $n = m$ this must equal $2\Phi(n) - 1$ (T4), and the code confirms it, e.g. 607 927 104 783 for $n = 10^6$.

```cpp
// coprime pairs in n x m rectangle = sum_d mu(d)*(n/d)*(m/d)
int Mu[MAXN];                        // prefix sums of mu
void build_mu_prefix(int n) { for (int i = 1; i <= n; i++) Mu[i] = Mu[i - 1] + mu[i]; }
ll coprime_rect(int n, int m) {
    ll ans = 0;
    int lim = min(n, m);
    for (int l = 1, r; l <= lim; l = r + 1) {
        r = min(n / (n / l), m / (m / l));
        ans += (ll)(Mu[r] - Mu[l - 1]) * (n / l) * (m / l);
    }
    return ans;
}
```

Sum of lcm **over all pairs.** Write $i = ga$, $j = gb$ with $\gcd(a,b) = 1$; then $\operatorname{lcm}(i,j) = g\,ab$. So

$$\sum_{i,j\le n} \operatorname{lcm}(i,j) = \sum_{g=1}^{n} g \cdot F\left(\left\lfloor \frac{n}{g} \right\rfloor\right), \qquad F(m) = \sum_{\substack{a,b\le m \\ \gcd(a,b)=1}} ab.$$

To compute F use μ again: substituting $a = da'$, $b = db'$ gives $F(m) = \sum_d \mu(d)\, d^2\, T(\lfloor m/d \rfloor)^2$ with $T(x) = \frac{x(x+1)}{2} = 1 + 2 + \cdots + x$. With prefix sums of $\mu(d)d^2$ and block loops, the total cost is about $O(n^{3/4})$.

Examples. $\sum_{i,j\le 3} \operatorname{lcm}(i,j) = 28$, $n = 4$: 72, $n = 10$: 2127.

```cpp
// sum_{i,j<=n} lcm(i,j) = sum_g g*F(n/g)        (mod MODP)
// F(m) = sum_{a,b<=m, coprime} a*b = sum_d mu(d) d^2 T(m/d)^2,  T(x)=x(x+1)/2
ll S2[MAXN];                          // prefix sums of mu(d)*d^2 (mod MODP)
void build_S2(int n) {
    for (int d = 1; d <= n; d++)
        S2[d] = ((S2[d - 1] + mu[d] * (ll)d % MODP * d) % MODP + MODP) % MODP;
}
ll F(ll m) {
    ll res = 0;
    for (ll l = 1, r; l <= m; l = r + 1) {
        r = m / (m / l);
        ll t = (m / l) * (m / l + 1) / 2 % MODP;
        res = (res + (S2[r] - S2[l - 1] + MODP) % MODP * (t * t % MODP)) % MODP;
    }
    return res;
}
ll lcm_all_pairs(ll n) {
    ll ans = 0;
    for (ll l = 1, r; l <= n; l = r + 1) {
        r = n / (n / l);
        ll gs = (l + r) * (r - l + 1) / 2 % MODP;      // g runs over [l, r]
        ans = (ans + gs * F(n / l)) % MODP;
    }
    return ans;
}
```

8.4 Three convolution identities worth knowing

Let $\mathbf{1}(n) = 1$, $\mathrm{id}(n) = n$, $\varepsilon(n) = [n = 1]$. Then

$$\mathbf{1} * \mu = \varepsilon \ \text{(Section 4, P11)}, \qquad \mathbf{1} * \varphi = \mathrm{id} \ \text{(P7)}, \qquad \mu * \mathrm{id} = \varphi \ \text{(P11)}.$$

Almost every formula in this guide is one of these three, with a sum swapped or a block loop applied.

8.5 Closed forms for multiplicative sums

Because φ is multiplicative, divisor sums of φ-based functions are multiplicative, so you only need prime powers:

$$\sum_{i=1}^{n} \gcd(i,n): \quad S(p^k) = (k+1)p^k - kp^{k-1}, \qquad \sum_{d \mid n} d\,\varphi(d): \quad G(p^k) = \frac{p^{2k+1}+1}{p+1}.$$

Example: $G(12) = G(4)\,G(3) = \frac{2^5+1}{3} \cdot \frac{3^3+1}{4} = 11 \cdot 7 = 77$, the same 77 as in Q8. These give $O(\sqrt{n})$ evaluation of Q7 and Q8 for a single huge n.

8.6 Very large numbers

For n up to 10^{18} (for example, $\varphi(n)$ in a modular-inverse problem), factor with Pollard's rho + Miller–Rabin and apply P3. There is no known faster way: computing $\varphi(n)$ is as hard as factoring n.

Further reading: the Codeforces blog "[Tutorial] Euler's phi function, its properties, and how to compute it" (codeforces.com/blog/entry/106851).

9 Cheat sheet

9.1 All the formulas on one page

Name	Formula
Definition	$\varphi(n) = \#\{1 \le k \le n : \gcd(k,n) = 1\},\ \varphi(1) = 1$
Prime power	$\varphi(p^k) = p^{k-1}(p-1)$
Product formula	$\varphi(n) = n\prod_{p\mid n}(1-\frac{1}{p})$
Multiplicative	$\gcd(m,n) = 1 \Rightarrow \varphi(mn) = \varphi(m)\varphi(n)$
General product	$\varphi(mn) = \varphi(m)\varphi(n)\frac{g}{\varphi(g)},\ g = \gcd(m,n);\ \varphi(n^k) = n^{k-1}\varphi(n)$
Parity	$\varphi(n)$ even for $n \ge 3$; $\varphi(2n) = \varphi(n)$ (n odd), $2\varphi(n)$ (n even)
Sum of coprimes	$\sum_{k\le n,\ (k,n)=1} k = \frac{n\varphi(n)}{2}\ \ (n \ge 2)$
Divisor sum	$\sum_{d\mid n}\varphi(d) = n$
Count by gcd	$\#\{k \le n : \gcd(k,n) = d\} = \varphi(n/d)$
Pillai	$\sum_{k\le n}\gcd(k,n) = \sum_{d\mid n} d\,\varphi(n/d);\ S(p^k) = (k+1)p^k - kp^{k-1}$
LCM sum	$\sum_{k\le n}\operatorname{lcm}(k,n) = \frac{n}{2}(1 + \sum_{d\mid n} d\varphi(d))$
Gcd as a sum	$\gcd(a,b) = \sum_{d\mid a,\,d\mid b}\varphi(d)$
Gcd sum on a rectangle	$\sum_{i\le n,\,j\le m}\gcd(i,j) = \sum_d \varphi(d)\lfloor\frac{n}{d}\rfloor\lfloor\frac{m}{d}\rfloor$
Coprime pairs	$\#\{a \le b \le n \text{ coprime}\} = \Phi(n);$ ordered: $2\Phi(n) - 1$
Pairs with gcd = g	$2\Phi(\lfloor n/g\rfloor) - 1$
Block identity	$\sum_d \varphi(d)\lfloor\frac{n}{d}\rfloor = \frac{n(n+1)}{2}\ \Rightarrow$ Du sieve
Möbius form	$\varphi(n) = \sum_{d\mid n}\mu(d)\frac{n}{d}$
Euler	$\gcd(a,m) = 1 \Rightarrow a^{\varphi(m)} \equiv 1 \pmod m;$ inverse $a^{\varphi(m)-1}$
Extended Euler	$b \ge \log_2 m \Rightarrow a^b \equiv a^{(b \bmod \varphi(m))+\varphi(m)} \pmod m$
Necklaces (rotations)	$\frac{1}{n}\sum_{d\mid n}\varphi(d)K^{n/d}$
Elements of order d	$\varphi(d)$ in a cyclic group, $d \mid$ group size; primitive roots mod p: $\varphi(p-1)$
Average	$\Phi(n) \approx \frac{3}{\pi^2}n^2$

9.2 Which code, and how fast?

Function	Section	Time	Notes
`phi_single(n)`	3.2	$O(\sqrt{n})$	$n \le 10^{12}$
`phi_sieve(N)`	3.3	$O(N \log\log N)$	only φ
`linear_sieve(N)`	3.4	$O(N)$	φ, μ, primes
`phi_segment(L,R)`	3.5	window $\times \log\log$	$R \le 10^{12}$
`inv_mod`, `power`	5.2	$O(\sqrt{m} + \log m)$	$\gcd(a,m) = 1$
`pow_huge`	5.4	$O(\sqrt{m} + \lvert b\rvert)$	any a, m; b as string
`tower`	5.5	$O(\log^2 m)$	stop when modulus is 1
`primitive_root(p)`	5.6	$O(\sqrt{p}\ + \ldots)$	p prime
`build_pillai`	Q7	$O(N \log N)$	$\sum \gcd(i,n)$ for all n
`build_lcm`, `lcm_sum`	Q8	$O(N \log N)$, $O(1)$	$\sum \operatorname{lcm}(i,n)$
`sum_gcd_all_pairs/rect`	Q9, Q10	$O(\sqrt{n})$ per query	needs Φ
`pairs_gcd_prime`	Q12	$O(\pi(n))$	needs Φ and primes
`count_coprime`, `kth_coprime`	Q4	$O(\sqrt{n} + 2^{\omega(n)})$	range counting
`necklaces`	Q16	$O(d(n)\sqrt{n})$	$n \le 10^9$
`Phi_du`	8.1	$O(n^{2/3})$	$n \le 10^{10}$–10^{11}
`inverse_phi`	8.2	small DFS	all n with $\varphi(n) = k$
`coprime_rect`, `lcm_all_pairs`	8.3	$O(\sqrt{n})$ / $O(n^{3/4})$	uses μ

9.3 Mistakes to avoid

1. $\varphi(1) = 1$. Formulas such as $\frac{n\varphi(n)}{2}$ (sum of coprimes) are for $n \geq 2$. Check what the problem means for $n = 1$.
2. Multiplicativity (P2) needs **coprime** arguments. Otherwise use P4.
3. Euler's theorem and "reduce the exponent mod $\varphi(m)$" need $\gcd(a, m) = 1$. Otherwise use the extended rule with $+\varphi(m)$ (and exact small exponents).
4. **Overflow.** $\Phi(10^{10}) > 2^{63}$; $\sum \operatorname{lcm}(i, n)$ is about $n^3/4$; products like $n \cdot \varphi(n)$ near 10^{18}. Use `__int128`, or reduce modulo a prime before multiplying.
5. Dividing by 2 modulo a prime: multiply by the inverse of 2, or (better) take the factor out of an even number before the multiplication.
6. Ordered versus unordered pairs; the pair $(1, 1)$ is counted once in $\Phi(n)$ and also on the diagonal.
7. The block loop $r = n/(n/l)$; with two limits take the minimum of the two r's (Q10).
8. The linear sieve for $N = 10^8$ may not fit in memory. Use a segmented sieve or the Du sieve.
9. Power towers: stop as soon as the modulus becomes 1, and keep the "$\geq m$" flag (the MOD trick).
10. Divisor/prime counts are tiny: $n \leq 10^9$ has ≤ 9 distinct primes and ≤ 1344 divisors. Brute-force over subsets or divisors is fine.

9.4 How to attack a new problem

1. Underline every **gcd, lcm, coprime, fraction, cycle, rotation, inverse, huge exponent**. Each one is a signal for φ.
2. One fixed n and a sum over $i \leq n$? → T1 / P8 (divisors and $\varphi(n/d)$).
3. Two free variables i, j? → T2 or T3, then prefix sums of φ and the block loop.
4. "How many coprime…"? → $\Phi(n)$; if the shape is a rectangle, use μ.
5. Modular arithmetic with a composite modulus, or a power with a huge exponent? → Euler and its extension.
6. Pick the algorithm from the constraints: single number (trial division), $N \leq 10^7$ (linear sieve), $N \leq 10^{10}$ (Du sieve), window at 10^{12} (segmented sieve).
7. Test against a brute-force solution for $n \leq 50$. Every formula in this guide was checked this way before it was printed.

9.5 Practice problems

Topic	Problems
Compute φ	UVa 10299 "Relatives"; SPOJ ETF "Euler Totient Function"
$\Phi(n)$, fractions	POJ 2478 "Farey Sequence"; Project Euler 72; LightOJ 1007 "Mathematically Hard"
gcd sums	UVa 11424 "GCD Extreme (I)"; UVa 11426 "GCD Extreme (II)"
lcm sums	SPOJ LCMSUM "LCM Sum"
prime gcd pairs	SPOJ PGCD "Primes in GCD Table" (needs the Möbius step of Section 8.3)
Euler's theorem	Codeforces 906D "Power Tower"
φ of products	Codeforces 1114F "Please, another Queries on Array?"
Extremes of $n/\varphi(n)$	Project Euler 69, Project Euler 70

If you remember only one thing: $\sum_{d|n} \varphi(d) = n$. *Everything else is this sentence in disguise.*